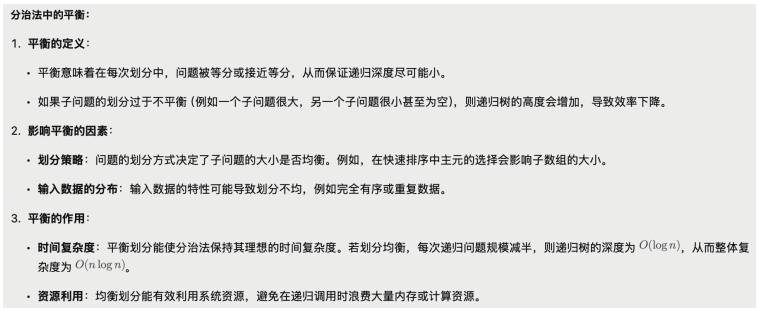
东南大学 2022

一、 判断题（共12分，每小题4分）

(1) 简述快速排序中划分过程的作用。

(2) 使用快速排序算法针对数组进行升序排序，以最右边元素作为主元，请写出针对（600, 500, 400, 300, 200, 100）这个输入实例进行排序的执行过程。

(3) 简述分治法与平衡的关系，并阐述你对快速排序算法中平衡的理解。



分治法中的平衡：

1. **平衡的定义**：
   - 平衡意味着在每次划分中，问题被等分或接近等分，从而保证递归深度尽可能小。
   - 如果子问题的划分过于不平衡（例如一个子问题很大，另一个子问题很小甚至为空），则递归树的高度会增加，导致效率下降。
2. **影响平衡的因素**：
   - **划分策略**：问题的划分方式决定了子问题的大小是否均衡。例如，在快速排序中主元的选择会影响子数组的大小。
   - **输入数据的分布**：输入数据的特性可能导致划分不均，例如完全有序或重复数据。
3. **平衡的作用**：
   - **时间复杂度**：平衡划分能使分治法保持其理想的时间复杂度。若划分均衡，每次递归问题规模减半，则递归树的深度为 $O(\log n)$，从而整体复杂度为 $O(n \log n)$。
   - **资源利用**：均衡划分能有效利用系统资源，避免在递归调用时浪费大量内存或计算资源。



快速排序中的平衡：

1. **划分过程**：
   - 快速排序选择一个主元（pivot），将数组分为两部分：
     - 左子数组：小于主元的元素。
     - 右子数组：大于主元的元素。
   - 然后递归地对左右子数组进行排序。
2. **平衡的影响**：
   - **平衡划分（最优情况）**：
     - 如果每次划分后，左右子数组的大小接近相等（即接近 $n/2$），递归树的高度为 $O(\log n)$，总时间复杂度为 $O(n \log n)$。
   - **不平衡划分（最坏情况）**：
     - 如果每次主元选择不当，例如总是选择最大值或最小值作为主元，划分会非常不平衡，一个子数组为空，另一个子数组接近 $n-1$ 个元素。此时递归树的高度为 $O(n)$，时间复杂度退化为 $O(n^2)$。
3. **主元选择与平衡**：
   - **随机化主元**：通过随机选择主元，可以避免特定输入（如完全有序数组）导致的不平衡。
   - **三数取中法**：选取数组中三个随机位置的元素，取中间值作为主元，可以提高划分的均衡性。
   - **平衡性的实质**：快速排序的性能核心在于保证划分尽可能接近平衡，即划分后子数组的大小相差不大。
4. **空间复杂度与平衡**：
   - 快速排序的空间复杂度与递归树的高度有关。
   - 平衡的划分能够使递归树高度为 $O(\log n)$，从而保证空间复杂度最优。

二、 （共12分，每小题4分）

针对（a）、（b）和（c）三式，请给出其紧确的渐近界。

(a) $T(n)=8T(n/8)+n$；(b)$T(n)=4T(n/2)+n$；(c)$T(n)=9T(n/3)+n^3$。

三、 （共10分）

设A是长为n的数组，存放n个互不相同的正整数。请设计一个平均时间复杂度为O(n)的算法，求不在 A 中出现的最小的自然数。

四、 （共10分）

设 A 是长为 n 的数组，首地址为 1，存放 n 个可以相同的整数。A 的一个子段是指二元组<a, b>, 满足 1<=a<=b<=n。子段<a, b>的和是指 A[a]+A[a+1]+...+A[b]。A 的一个最大子段是指其和不小于任何一个 A 的子段和的子段。请设计一个最坏时间复杂度为 O(n)的算法，求 A 的最大子段的个数。

答：

1. **遍历数组，动态计算最大子段和**
   使用变量 s 记录当前子段的和，如果变量小于 0，从当前元素为起点，重新开始计算子段。使用 max_sum 记录目前最大的子段和。如果当下 s 大于等于 max_sum，则更新，并增加计数器 count+1。

五、 （共8分）

(1) 描述钱币兑换问题的一个有效贪心算法，目标是利用最少量的硬币，兑换某个指定的币值。假设有 4 种硬币，面值分别是 25， 10， 5， 1 分。证明你的算法的正确性。

(2) 给出一组硬币面值的示例，用来说明你前面设计的算法并非对任意硬币面值集合都适用。

答：（1）（2）

1. 按硬币面值从大到小排序，形成硬币列表 $C = [25, 10, 5, 1]$。
2. 初始化一个计数器 `coin_count = 0`，表示使用的硬币总数。
3. 对于每个硬币面值 c：
   - 使用尽可能多的该面值硬币，即计算当前面值可用硬币数量 $k = \lfloor S/c \rfloor$。
   - 将 k 计入总数：`coin_count += k`。
   - 更新剩余金额：$S = S\%c$。
4. 如果 $S = 0$，算法结束，输出 `coin_count`。

假设硬币面值为 [9, 6, 1]，需要兑换金额 $S = 11$。

贪心算法结果

1. 按贪心选择，首先选用面值最大的 9 分硬币，得到 1 枚 9 分硬币，剩余金额 $S = 2$。
2. 选择 1 枚 1 分硬币，总计硬币数为 2 + 1 = **3 枚**。

实际最优解

选用 2 枚 6 分硬币，总计硬币数为 **2 枚**。

原因

贪心算法在选择面值 9 时没有考虑到后续兑换的整体最优性，导致结果不是全局最优。

动态规划：$dp[x] = \min_{c \in C}\{dp[x-c]+1\}$

六、 （共10分，第（1）题4分，第（2）题6分）

(1) 简述在通常情况下，我们对随机算法的分类，说明这些不同种类的随机算法的特点。

(2) 请设计一个随机算法（用自然语言简单进行描述），它能在 n 个数据元素中找出第 k 小元素。简要分析你设计的算法的时间复杂度。

类型	是否保证正确性	运行时间是否随机	应用场景
Las Vegas	保证	随机	快速排序、图匹配
Monte Carlo	不保证，但高概率正确	固定	圆周率估算、最小割
哈希类随机算法	基于哈希的正确性	固定	数据流分析、去重
Sherwood	不保证	随机	模拟退火、优化复杂问题

随机数值算法

- 主要用于数值问题求解
- 算法的输出往往是近似解
- 近似解的精确度与算法执行时间成正比

Monte Carlo算法

- 主要用于求解需要准确解的问题
- 算法可能给出错误解
- 获得精确解概率与算法执行时间成正比

Las Vegas算法

- 一旦找到一个解，该解一定是正确的
- 找到解的概率与算法执行时间成正比
- 增加对问题反复求解次数，可是求解无效的概率任意小

Sherwood算法

- 一定能够求得一个正确解
- 确定算法的最坏与平均复杂性差别大时，加入随机性，即得到Sherwood算法
- 消除最坏行为与特定实例的联系

1. **随机选择一个主元 (pivot)**：
   - 从数组中随机选择一个元素作为主元 (pivot)。
2. **划分数组**：
   - 将数组分为三部分：
     - 小于主元的元素 (左子数组)。
     - 等于主元的元素。
     - 大于主元的元素 (右子数组)。
3. **判断第 k 小元素的位置**：
   - 计算左子数组的大小 m。
   - 如果 $k \le m$，第 k 小元素一定在左子数组中，递归在左子数组中查找第 k 小元素。
   - 如果 $k > m+1$，第 k 小元素在右子数组中，递归在右子数组中查找第 $k-m-1$ 小元素。
   - 如果 $k = m+1$，则主元就是第 k 小元素。
4. **递归直到找到答案**：
   - 不断缩小查找范围，直到找到第 k 小元素。

七、 （共10分，第（1）题4分，第（2）题6分）

(1) 简述回溯法和分枝限界法如何解决问题。

(2) 在一个儿童故事中，3 只狐狸和 3 只母鸡要过河，小船最多只能搭乘两个动物，在河的任何一边如果狐狸的数目超过了母鸡，狐狸就会把母鸡吃掉，在使用搜索算法寻找安全的过河方案

时，描述该问题的状态空间【**请注意，只需要描述状态空间的结点和有向边，不需要举例说明求解过程**】。

回溯法

- **思想**：
  - 回溯法是一种系统的搜索算法，用于解决约束满足问题（如排列、组合、图的着色等）。
  - 从初始状态出发，逐步扩展解空间树的节点，尝试各种可能的解。
  - 在搜索过程中，若当前解不满足问题约束，则回溯到上一步，尝试其他可能的路径。

分枝限界法

- **思想**：
  - 分枝限界法是一种求解优化问题的搜索算法，旨在在部分解空间中找到最优解。
  - 解空间以树的形式组织，算法逐步扩展分枝，同时通过限界函数剪枝，排除不能产生最优解的分枝。

- 设每种动物的数量为 F(狐狸数) 和 C(母鸡数)，节点状态可表示为：
(F_L, C_L, B, F_R, C_R)

状态空间的有向边

- 有向边表示从一个状态 (节点) 到另一个状态 (节点) 的合法移动。
- 合法的移动规则：
  1. 船上人数最多为 2，最少为 1。
  2. 每次移动必须改变船的位置。
  3. 移动后，在任意岸上，母鸡的数量不能少于狐狸的数量 (如果有母鸡)。
- 边可以表示为一个状态转移操作：
$(\mathrm{F_m},\ \mathrm{C_m})$
其中 F_m 和 C_m 表示移动的狐狸数和母鸡数，需满足 $F_m + C_m \leq 2$ 且 $F_m + C_m \geq 1$。

八、 （共16分，第（1）题12分，第（2）题4分）

(1) 说说你对 NP 完全问题、NP Hard 类问题、P 类问题和 NP 类问题的理解。

P 问题是在多项式时间内被确定型图灵机解决的问题，是高效的，时间复杂度在多项式级别

NP 类问题：在多项式时间内由非确定型图灵机验证解的问题。包括所有 P 类问题，但也可能包含一些未能证明可在多项式时间内解决的问题。常见问题如：旅行商问题（TSP）和 0/1 背包问题。

NP 完全问题：NP 完全问题是 NP 类中“最难”的问题。如果一个 NP 完全问题可以用多项式时间解决，那么所有 NP 类问题都可以用多项式时间解决

NP Hard 问题：可能不属于 NP 类（即可能无法在多项式时间内验证解的正确性）。

(2) 选择任意一个 NP 完全问题或 NP Hard 类问题，做简要描述。

(2) 任意一个NP完全问题：3-SAT问题

问题描述：

- 给定一个布尔公式，公式以合取范式 (CNF) 表示，每个子句是三个变量或它们的否定的析取。
$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge \cdots$
- **目标**：判断是否存在一种变量赋值，使得整个公式为真。

性质：

1. **属于NP类**：给出一个布尔变量的赋值，可以在多项式时间内验证公式是否为真。
2. **NP完全性**：可以通过多项式时间归约，将任何 NP 类问题转换为 3-SAT 问题。

实际意义：

- 3-SAT 问题广泛应用于电路设计、逻辑推理、人工智能等领域。
- 是研究 NP 完全问题的起点之一，许多其他问题都是通过对 3-SAT 的归约来证明其 NP 完全性。

九、 （共12分，第（1）题4分，第（2）题8分）

(1) 在实际应用中，对于有些问题，为何需要设计近似算法求解？

1. **问题的计算复杂性**：
   - 有些问题属于 **NP完全问题** 或 **NP Hard类问题**，用现有算法无法在多项式时间内找到最优解。
   - 例如，旅行商问题、装箱问题等。随着问题规模增加，求解时间会指数级增长，实际中往往不可接受。
2. **可接受的解精度**：
   - 对于实际应用，最优解和近似解之间的差异可能在可接受范围内。
   - 例如，某些情况下，99%的解已经足够满足需求。
3. **资源和时间限制**：
   - 在实际场景中，往往面临计算资源有限或实时性需求，无法容忍过长的求解时间。
4. **算法设计的简单性**：
   - 近似算法通常比最优解算法更简单，易于实现和推广。

(2) 设有 n 个物体，每个物体的体积不超过 1。另外，有足够多的、体积为 1 的箱子。箱子、物体均是长方体且截面相同。现在需要将物体装入箱子，请问如何装箱，使得所用箱子数最少？请设计算法，并做相应分析。

第一次适应，尝试放下，放的下就直接放入，直到放不下

贪心：按照物品体积由大到小排序，再使用第一次适应